

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Văn A - Trần Thị B

HƯỚNG DẪN TRÌNH BÀY BÁO CÁO
KHÓA LUẬN TỐT NGHIỆP/
THỰC TẬP TỐT NGHIỆP/
THỰC TẬP DỰ ÁN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng MM/YYYY

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Văn A - 1512090

Trần Thị B - 1512007

HƯỚNG DẪN TRÌNH BÀY BÁO CÁO
KHÓA LUẬN TỐT NGHIỆP/
THỰC TẬP TỐT NGHIỆP/
THỰC TẬP DỰ ÁN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN

PGS.TS. Hoàng Văn C

TS. Lê Thị D

Tp. Hồ Chí Minh, tháng MM/YYYY

Lời cam đoan

Tôi xin cam đoan đây là công trình nghiên cứu của tôi dưới sự hướng dẫn của ...

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Tôi xin chân thành cảm ơn ...

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó *(phải có chữ ký của giảng viên hướng dẫn)*.

Contents

Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vi
Mục lục	vii
List of Figures	x
List of Tables	xii
Tóm tắt	xii
1 Ngôn ngữ	1
2 Proposed System: Architecture, Methodology, and Implementation	2
2.1 System Architecture	2
2.1.1 Design goals	3
2.1.2 Three-zone reference architecture	4
2.1.3 End-to-end data flow	5

2.1.4	Hybrid deployment topology	7
2.1.5	Component map and interfaces	7
2.1.6	Key data contracts	8
2.1.7	Security and trust model	9
2.2	Proposed Methodology	9
2.2.1	Approach overview	10
2.2.2	AI-assisted generation and the regeneration loop . .	10
2.2.3	Pre-validation	12
2.2.4	Security evaluation and cross-source deduplication .	12
2.2.5	Risk-scoring engine (overview)	13
2.2.6	Deployment governance	14
2.2.7	Monitoring and the operational loop	14
2.3	Risk-Scoring Engine	14
2.3.1	Motivation and limitations of severity-only scoring .	15
2.3.2	Multi-factor formulation	15
2.3.3	Implemented additive scoring	16
2.3.4	Machine-learning code-risk model	18
2.3.5	Deduplication and explainability	19
2.3.6	Worked example	20
2.4	Implementation	20
2.4.1	Technology stack and repository layout	21
2.4.2	Generation layer	22
2.4.3	Security gate and scanner adapters	22
2.4.4	Orchestration and governance	24
2.4.5	Execution backbone	25
2.4.6	Phase 4 observability	26
2.4.7	Operator console and multi-project registry	26
3	Hướng dẫn sử dụng template	27
3.1	Trích dẫn tài liệu	27
3.2	Chèn mã nguồn	27

3.3	Hình ảnh	28
3.4	Bảng biểu	29
3.5	Công thức	29
	Danh mục công trình của tác giả	31
	Tài liệu tham khảo	32
	A Ngữ pháp tiếng Việt	34
	B Ngữ pháp tiếng Nôm	35

List of Figures

2.1	High-level three-zone architecture and end-to-end flow. Solid arrows are the forward path; the dashed arrow is the regeneration feedback loop.	5
2.2	Artifact lifecycle of a single request. The decision is computed on the synthesized template; a rejection feeds findings back into generation.	6
2.3	Hybrid deployment topology: AWS resources and on-premises nodes joined by an encrypted mesh, with on-premises nodes registered as SSM managed instances.	7
2.4	Regeneration loop as a flowchart. The loop exits on an accepting decision or when the attempt budget N is exhausted.	12
2.5	Security-evaluation pipeline: parallel scanners, normalization to a common schema, cross-source deduplication, and aggregation by the risk-scoring engine.	13
2.6	The additive risk-scoring engine: four orthogonal sub-scores are summed and the total is compared against two fixed thresholds to yield the decision.	18
2.7	Runtime module dependencies. Entry points drive orchestration, which drives generation, evaluation, and monitoring; all external processes pass through the single execution helper.	22

2.8	Hybrid pipeline interaction sequence. The orchestrator gates both sides with the same engine, deploys each side only on an accepting decision, and emits telemetry.	25
3.1	Hình ví dụ 1	29
3.2	Hình ví dụ 2	29

List of Tables

2.1	The three zones of the proposed architecture.	4
2.2	Top-level modules and their responsibilities.	8
2.3	Mapping from hybrid-cloud challenges to solution modules.	10
2.4	Severity-to-points mapping used by the gate.	17
2.5	Decision thresholds and their operational meaning.	18
2.6	Repository layout and the concern each directory implements.	21
2.7	Scanner adapters and the signal each contributes.	23
3.1	Bảng ví dụ 1	30

Tóm tắt

Giới hạn 200-300 từ, sử dụng văn phong học thuật, ngắn gọn, khách quan, viết liền mạch, không gạch đầu dòng.

Hướng dẫn trình bày tóm tắt:

1. Giới thiệu vấn đề/ngữ cảnh của vấn đề (khoảng 1 - 3 câu): Trình bày bối cảnh và vấn đề thực tế đang tồn tại/ khoảng trống nghiên cứu, tầm quan trọng của vấn đề nghiên cứu, lý do cần thực hiện đề tài.
2. Mục tiêu đề tài (1 câu): nêu cụ thể đề tài giải quyết/phát triển điều gì.
3. Phương pháp và giải pháp (1 - 3 câu): Trình bày phương pháp, công nghệ, công cụ, mô hình, giải pháp được đề xuất trong đề tài. Nêu những cải tiến nổi bật (nếu có). Nêu phương pháp thử nghiệm, đánh giá, bộ dữ liệu được sử dụng để đánh giá (nếu có)
4. Kết quả chính (1 - 2 câu): Trình bày ngắn gọn các kết quả nổi bật, có thể đưa số liệu cụ thể (độ chính xác, tốc độ xử lý, dung lượng dữ liệu, ...)
5. Kết luận (1 - 2 câu): Nêu ý nghĩa khoa học và thực tiễn (nếu có) của phương pháp/giải pháp hay kết quả của đề tài, khả năng áp dụng vào thực tế/đóng góp cho doanh nghiệp/người dùng/nghiên cứu sau này, ...

Chapter 1

Ngôn ngữ

Ngôn ngữ để viết và trình bày báo cáo khóa luận tốt nghiệp, Thực tập tốt nghiệp, Thực tập dự án tốt nghiệp (sau đây gọi chung là báo cáo) là tiếng Việt hoặc tiếng Anh.

Trường hợp chọn ngôn ngữ tiếng Anh để viết và trình bày báo cáo, sinh viên cần có đơn đề nghị, được giảng viên hướng dẫn (GVHD) đồng ý và nộp cho bộ phận Giáo vụ của Khoa vào thời điểm đăng ký đề tài để xin ý kiến.

Báo cáo viết và trình bày bằng tiếng Anh phải có bản tóm tắt viết bằng tiếng Việt.

Chapter 2

Proposed System: Architecture, Methodology, and Implementation

This chapter presents the proposed SysSecOps system for a hybrid cloud environment. It first describes the system architecture (Section 2.1), then the methodology of the proposed process in reproducible detail (Section 2.2), the risk-scoring engine that turns heterogeneous tool output into a single decision (Section 2.3), and finally how the methodology is realised in software (Section 2.4).

2.1 System Architecture

This section presents the architecture of the proposed SysSecOps system for a hybrid cloud environment. The design places a quantitative, multi-scanner *security gate* between the synthesis and the deployment of Infrastructure-as-Code (IaC), so that no change is applied to either the public-cloud or the on-premises side until it has been statically analysed, costed, checked against live compliance state, and assigned a single risk score. The same engine governs both sides of the hybrid environment, which gives the process a uniform and reproducible security behaviour.

2.1.1 Design goals

The architecture is driven by the research objectives stated in Chapter 1 and refined into the following concrete design goals:

- **Gate before deploy.** Security evaluation must occur on the *synthesized* artifact (the CloudFormation template or the resolved Ansible play), not merely on the higher-level source, because the synthesized artifact is the last representation before any resource is provisioned.
- **One engine for the whole hybrid.** The AWS (public) side and the on-premises (private) side must be scored with identical severity weights, deduplication rules, and decision thresholds, so that decisions are comparable across the trust boundary.
- **Graceful degradation.** A missing scanner binary or absent cloud credentials must never crash the pipeline; the affected component degrades to a well-defined “skipped” status while the gate still produces a decision.
- **Auditability.** Every decision (*pass*, *review*, *reject*) is persisted as an immutable record together with the acting identity, enabling post-hoc review.
- **Feedback-driven remediation.** When code is generated, gate findings are injected back into the generation prompt so that unsafe code is automatically regenerated rather than merely blocked.
- **Observability.** Every deployed target is continuously monitored and every gate decision is published as telemetry for dashboards and automated remediation.

2.1.2 Three-zone reference architecture

The system is organised into three cooperating zones, summarised in Table 2.1. Zone 1 produces IaC; Zone 2 evaluates and gates it; Zone 3 provisions the hybrid infrastructure and continuously observes it. The risk-scoring engine sits at the boundary of Zone 1 and Zone 2: it consumes scanner output and emits the decision that governs whether code advances to deployment or returns to Zone 1 for regeneration.

Table 2.1: The three zones of the proposed architecture.

Zone	Name	Responsibility
Zone 1	AI + IaC local development	Generate IaC from a natural-language prompt; local validation (<code>cdk synth</code> , <code>cdk diff</code>).
Zone 2	IaC security gate	Multi-scanner analysis, cross-source deduplication, risk scoring, and the deploy decision.
Zone 3	Hybrid infrastructure and operations monitoring	Provision AWS and on-premises resources; publish telemetry; run the drift/remediation operational loop.

Figure 2.1 shows the three zones and the principal flow of control between them. The solid arrows denote the forward path of a change from prompt to deployed and monitored infrastructure; the dashed arrow denotes the remediation feedback that returns rejected generated code to Zone 1.

The three zones are intentionally loosely coupled. Zone 1 knows nothing about the internal scoring rules of Zone 2 — it only consumes the machine-readable findings that Zone 2 emits. Zone 3 knows nothing about how a decision was reached — it only consumes the decision event. This decoupling is what allows the generation backend, the set of scanners, and the monitoring stack to evolve independently, and it is the reason the same

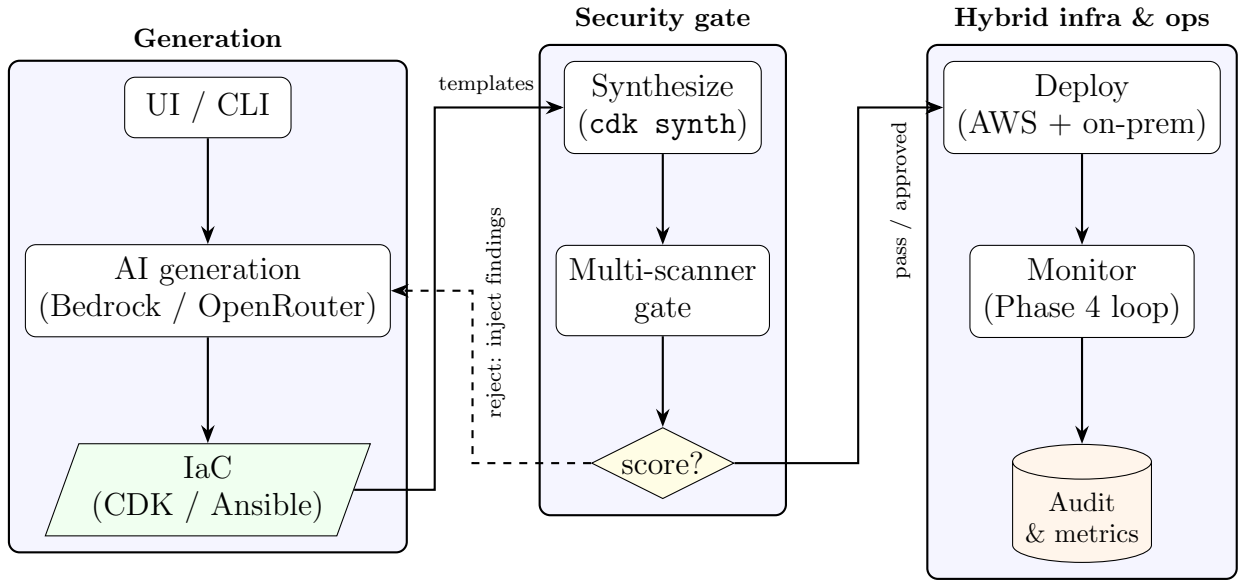


Figure 2.1: High-level three-zone architecture and end-to-end flow. Solid arrows are the forward path; the dashed arrow is the regeneration feedback loop.

gate can serve both the public and the private side without modification.

2.1.3 End-to-end data flow

A single request flows through the system as an evolving chain of artifacts:

1. A natural-language prompt (or an existing, “bring-your-own” project) enters the system through the command-line interface or the operator console.
2. Zone 1 produces IaC: a synthesizable AWS Cloud Development Kit (CDK) Python application and/or a set of Ansible playbooks.
3. The IaC is synthesized: `cdk synth` emits CloudFormation templates, while `ansible-playbook --syntax-check` validates the playbooks.
4. Zone 2 evaluates the synthesized artifact with several independent

scanners, normalises their outputs into a common finding schema, deduplicates across sources, and computes a risk score.

5. The decision function maps the score to *pass*, *review*, or *reject*. The full gate report is persisted, and an approval or rejection record is written.
6. On *pass* (or an approved *review*), the change is deployed; on *reject*, generated code is optionally regenerated with the findings injected back into the prompt.
7. Zone 3 records the outcome, publishes metrics and events, and attaches post-deployment security monitoring.

Figure 2.2 traces the same request as a chain of artifacts, making explicit how each stage transforms its input into a more concrete representation. The important property is that the security decision is taken on the *synthesized template*, not on the prompt or the high-level source: this is the last artifact before provisioning and therefore the earliest point at which the exact resources, their properties, and their relationships are fully known.

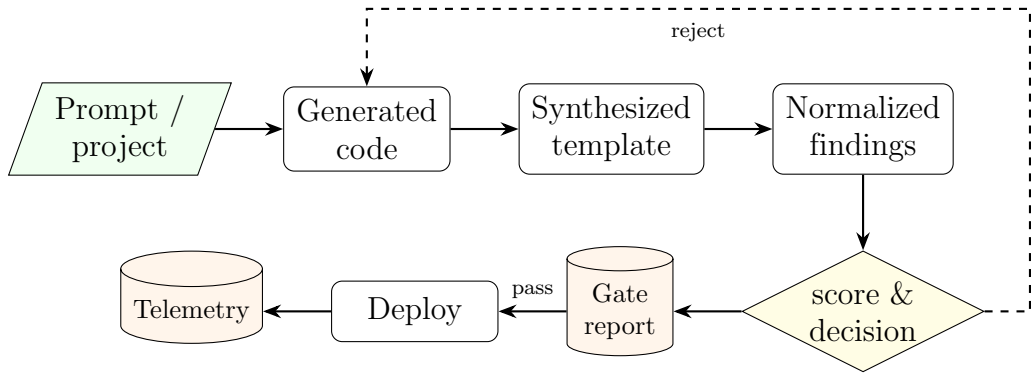


Figure 2.2: Artifact lifecycle of a single request. The decision is computed on the synthesized template; a rejection feeds findings back into generation.

2.1.4 Hybrid deployment topology

The public side is provisioned on Amazon Web Services (AWS) through CDK-synthesized CloudFormation. The private side consists of on-premises Linux nodes managed by Ansible. The two sides are connected through an encrypted mesh overlay (a WireGuard-based virtual private network), and the on-premises nodes are registered as AWS Systems Manager (SSM) managed instances so that their compliance state is visible to the same operational loop that watches the cloud side. No public inbound administrative access is required: the control plane traverses the encrypted tunnel, which keeps the hybrid trust boundary small and observable. Figure 2.3 depicts the topology.

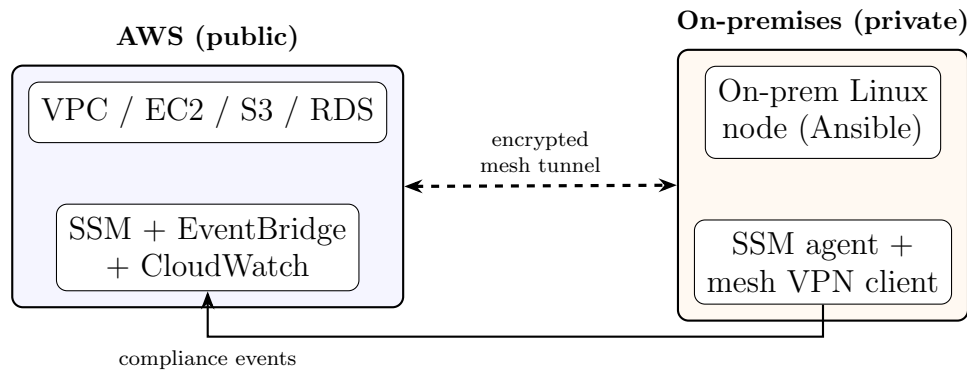


Figure 2.3: Hybrid deployment topology: AWS resources and on-premises nodes joined by an encrypted mesh, with on-premises nodes registered as SSM managed instances.

2.1.5 Component map and interfaces

Table 2.2 lists the top-level software modules and their responsibilities. The modules communicate through stable contracts rather than shared internal state, which keeps the generation, evaluation, orchestration, and monitoring concerns decoupled.

Table 2.2: Top-level modules and their responsibilities.

Module	Responsibility
AIgen/	Zone 1 generation: provider-abstracted code generation (Bedrock / OpenRouter) and the CDK regeneration loop.
Eval/	Zone 2 gate: template and playbook analysis, scanner adapters, deduplication, risk scoring, and report assembly.
ExecComponent/	Subprocess execution backbone with a stable result contract.
pipeline/	Orchestration and governance: synth-gate-diff-deploy, the approval audit trail, credentials, and the Phase 4 emitters.
Monitor/	Phase 4 observability: CloudTrail, CloudWatch metrics and dashboards, the operational-loop Lambda, and on-premises SSM registration.
scripts/	Command-line entry points for the CDK-only and the hybrid flows.
ui/	Streamlit operator console and the multi-project registry.
RiskScoringCitation/	Training and evaluation code for the machine-learning code-risk model consumed by the gate.

2.1.6 Key data contracts

Three contracts hold the system together and are deliberately kept stable so that independent consumers (the CLI, the console, the dashboards) do not break:

- **Gate report.** A JSON document containing the run identifier, timestamp, numeric **score**, **decision**, human-readable **message**, a per-component **components** breakdown, the deduplicated **findings**, and a **scanner_status** map that records whether each scanner ran,

was skipped, or was unavailable.

- **Execution result.** Every external command returns exactly `{return_code, output}`, with standard output and standard error merged to preserve event ordering for the console and the logs.
- **Gate-decision event.** A compact event (target name, score, decision, timestamp) is persisted to SSM Parameter Store and published to Amazon EventBridge, keyed by the target name (a stack name for CDK, or `ansible-<node>` for the on-premises side) so that the two sides never collide.

2.1.7 Security and trust model

Credentials are never passed as command-line arguments; they are resolved through the standard provider chains and injected into subprocesses through the environment. AWS access follows the default credential chain with a single-sign-on fallback, and generation-provider keys are read from the environment. Deployed workloads follow the principle of least privilege, and each privileged action is recorded in an append-only audit trail. The uniform graceful-degradation contract guarantees that the absence of a tool or a credential weakens visibility but never silently disables the gate.

2.2 Proposed Methodology

This section describes the methodology of the proposed process in enough detail that the pipeline could be reimplemented from the description. The approach reuses an established security-automation baseline and extends it with three additions tailored to hybrid cloud: (i) an AI-assisted generation stage with a closed remediation loop, (ii) a unified risk-scoring engine that aggregates heterogeneous tool findings into a single decision,

and (iii) a uniform application of the same gate to both the public and the private side of the environment.

2.2.1 Approach overview

Each hybrid-cloud challenge identified in Chapter 1 is mapped to a concrete module that addresses it. Table 2.3 makes this mapping explicit and provides the traceability from problem to solution that motivates the remainder of the section.

Table 2.3: Mapping from hybrid-cloud challenges to solution modules.

Challenge	Module / mechanism
Weak security linkage between public and private cloud	One gate and one risk engine applied to both the CDK and the Ansible side.
Absence of a clear, repeatable process	The synth–gate–decide–deploy pipeline with fixed thresholds and audit records.
Fragmented tooling with inconsistent output	Scanner adapters that normalise findings into a common schema, followed by cross-source deduplication.
Misconfiguration risk amplified by fast AI generation	The generate–gate–regenerate feedback loop that injects findings back into the prompt.
Context-free prioritisation	A risk score that combines severity with cost, live compliance, and a machine-learning code-risk signal.

2.2.2 AI-assisted generation and the regeneration loop

Zone 1 generates synthesizable CDK Python code from a natural-language prompt. Code generation is placed behind a provider abstraction so that

two backends — a managed foundation-model service and a hosted inference API — are interchangeable behind a single provider flag and share an identical generation contract. Model defaults are centralised and can be overridden explicitly.

Generation alone does not guarantee secure output, so it is wrapped in a closed loop. The loop generates code, synthesizes it, gates it, and — if the gate rejects — rebuilds the prompt with the machine-readable findings injected before retrying, up to a bounded number of attempts. Each attempt is persisted for reproducibility under `logs/cdk_regen/<run_id>/attempt_<N>/` (the prompt snapshot, the generated code, the synthesis output, and the gate report), and the successful attempt is promoted to a `passed/` folder. Algorithm 1 formalises the loop.

Algorithm 1 Generate–synthesize–gate–regenerate loop

Require: prompt q , maximum attempts N

Ensure: gated CDK application, or failure after N attempts

```

1:  $findings \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $N$  do
3:    $code \leftarrow \text{GENERATE}(q, findings)$ 
4:   write  $code$  to the CDK application
5:    $t \leftarrow \text{SYNTH}(code)$  ▷ emit CloudFormation templates
6:   if SYNTH failed then
7:      $findings \leftarrow$  synthesis error
8:     continue
9:    $report \leftarrow \text{GATE}(t)$ 
10:  if  $report.decision \neq reject$  then
11:    return  $report$  ▷ pass or review
12:   $findings \leftarrow report.findings$  ▷ injected into the next prompt
13: return failure

```

Figure 2.4 presents the same loop as a flowchart, emphasising the two exit conditions: an accepting decision (*pass* or *review*) leaves the loop with a gated application, whereas exhausting the attempt budget N leaves it with a failure that surfaces to the operator.

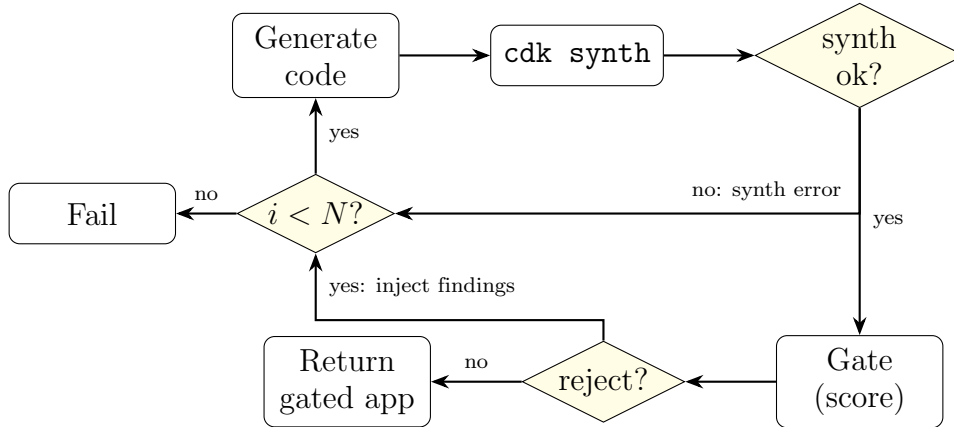


Figure 2.4: Regeneration loop as a flowchart. The loop exits on an accepting decision or when the attempt budget N is exhausted.

2.2.3 Pre-validation

Before any security analysis, each side is validated syntactically. On the CDK side, `cdk synth` must succeed (otherwise the loop treats the failure as feedback), and `cdk diff` previews the change. On the Ansible side, `ansible-playbook --syntax-check` validates the plays and `ansible-playbook --check --diff` performs a dry run. This staging (*validate* $\rightarrow$ *dry run* $\rightarrow$ *deploy*) is symmetric across the two sides and ensures the gate only ever scores artifacts that are at least well formed.

2.2.4 Security evaluation and cross-source deduplication

Zone 2 analyses the synthesized artifact with several independent scanners, each wrapped in an adapter that normalises its output into a common finding schema with the fields `severity`, `source`, `message`, `resource_id`, `template`, and `category`. Because several tools frequently report the same underlying issue (for example, an open administrative port may be flagged by both a heuristic check and a policy scanner), the findings are deduplicated by the key (`resource_id`, `template`, `category`). When duplicates collide, the highest severity is retained and the source labels are merged.

Deduplication is essential: without it, redundant reports would inflate the score and distort the decision. Figure 2.5 shows the analysis pipeline: the synthesized artifact fans out to the parallel scanner adapters, whose normalized findings are merged, deduplicated, and passed to the risk-scoring engine.

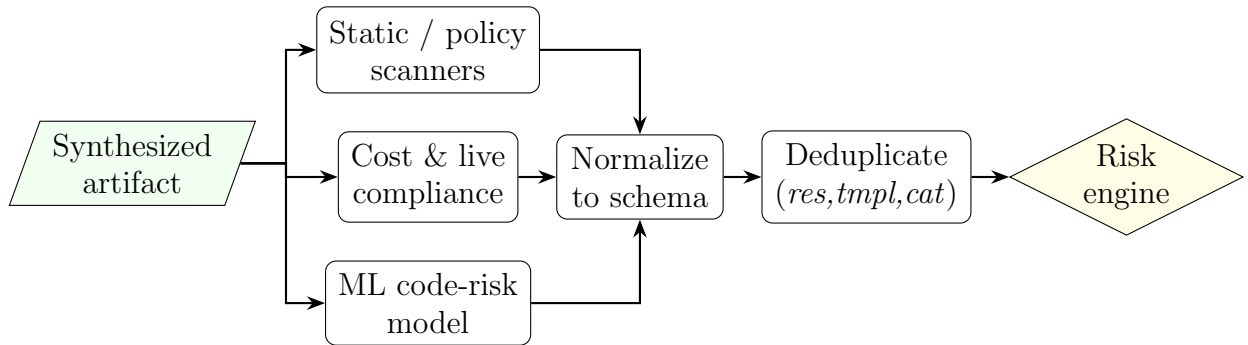


Figure 2.5: Security-evaluation pipeline: parallel scanners, normalization to a common schema, cross-source deduplication, and aggregation by the risk-scoring engine.

2.2.5 Risk-scoring engine (overview)

The deduplicated findings, together with the cost, live-compliance, and machine-learning code-risk signals, are aggregated by the gate into a single numeric score that is then mapped to one of three decisions — *pass*, *review*, or *reject*. Because this aggregation is the analytical heart of the proposed process, it is presented in full in Section 2.3, which develops the multi-factor formulation, the additive scoring implementation, the machine-learning code-risk model, the deduplication-driven explainability, and a worked example. Here it suffices to note that the score is an *additive* combination of weighted findings, banded cost, live compliance violations, and a bounded machine-learning contribution, and that the decision thresholds are fixed so that identical inputs always yield identical decisions on both sides of the hybrid environment.

2.2.6 Deployment governance

The decision governs deployment. A *pass* deploys automatically. A *review* requires an explicit approval, which is recorded together with the approver’s cloud identity, the run identifier, the score, and the decision. A *reject* never deploys; when the code was generated, the regeneration loop is invoked, and otherwise a rejection record is written for the author to act on. Approval and rejection records are append-only, which yields the immutable audit trail required by the design goals.

2.2.7 Monitoring and the operational loop

After a successful deployment, Zone 3 attaches post-deployment monitoring and closes an operational loop. Each gate decision is persisted to SSM Parameter Store, published as an EventBridge event, and emitted as CloudWatch metrics and structured logs. An event-driven function reacts to compliance drift and rollback events, enabling automated remediation. Because the on-premises nodes are registered as managed instances, their compliance events flow through the same loop, so the entire hybrid environment is observed uniformly rather than as two disconnected halves.

2.3 Risk-Scoring Engine

The risk-scoring engine is the analytical core of the proposed process: it converts the heterogeneous output of several independent security tools into one number and one decision. This section motivates why a single, purpose-built score is needed (Section 2.3.1), states the multi-factor formulation (Section 2.3.2), describes the additive scoring actually implemented in the gate (Section 2.3.3), details the machine-learning code-risk model (Section 2.3.4), explains how deduplication yields explainability (Section 2.3.5), and closes with a worked example (Section 2.3.6).

2.3.1 Motivation and limitations of severity-only scoring

Industry practice prioritises vulnerabilities almost exclusively by technical severity, typically a Common Vulnerability Scoring System (CVSS) base score. For infrastructure changes in a hybrid cloud this is insufficient for three reasons:

- **Severity ignores blast radius and cost.** An open security-group rule on an internet-facing load balancer and the same rule on an isolated subnet may share a severity label yet differ enormously in exposure. Likewise, a change that quietly provisions expensive resources carries an operational risk that no security-only score captures.
- **Single tools have blind spots.** A policy scanner, a template linter, a cost estimator, a live-compliance service, and a code-level model each see a different slice of the same change. Relying on any one of them yields systematic false negatives.
- **AI generation shifts the risk profile.** When code is produced quickly by a language model, the dominant failure mode is not a novel exploit but a plausible misconfiguration. A signal that estimates the *probability* that generated code is insecure is therefore as valuable as any individual rule.

The engine addresses these limitations by combining several orthogonal signals into one additive score with a fixed, auditable decision boundary, so that prioritisation reflects severity *and* exposure, cost, live compliance, and a learned code-risk estimate.

2.3.2 Multi-factor formulation

Conceptually, the risk of a change is a weighted combination of five factors:

- **Severity** (S): the technical seriousness of each finding.
- **Exposure** (X): how reachable the affected resource is (for example, public versus private).
- **Exploitability** (E): how readily a finding can be abused.
- **Confidence** (C): how certain the detecting tool is, which discounts low-confidence heuristics.
- **Cost** (K): the financial blast radius of the change.

A general multi-factor risk of a change with finding set F can be written as

$$R = \sum_{f \in F} \alpha S_f \cdot X_f \cdot E_f \cdot C_f + \beta K, \quad (2.1)$$

where α and β balance the finding-driven term against the cost term. This formulation is the design intent; the implemented engine (next subsection) is a deliberately simplified, *additive* realisation of it that keeps the score transparent and reproducible while folding exposure, exploitability, and confidence into the calibrated severity weights and the live-compliance term.

2.3.3 Implemented additive scoring

The gate implements Equation 2.1 as an additive sum of four components that are easy to explain and cheap to compute. Each deduplicated finding contributes points according to its severity (Table 2.4); a monthly cost increase contributes points in bands; each non-compliant live rule contributes a fixed number of points; and the machine-learning model contributes points proportional to its predicted probability of insecurity.

Formally, let F be the set of deduplicated findings, $\text{sev}(f)$ the severity of finding f , and $w(\cdot)$ the mapping in Table 2.4. Let Δ be the estimated

Table 2.4: Severity-to-points mapping used by the gate.

Severity	Points
Critical	20
High	10
Medium	5
Low	1

monthly cost increase in US dollars, $c(\Delta)$ the banded cost contribution, v the number of non-compliant live-compliance rules, and $p \in [0, 1]$ the predicted probability of insecurity from the machine-learning model. The total score S is

$$S = \underbrace{\sum_{f \in F} w(\text{sev}(f))}_{\text{severity}} + \underbrace{c(\Delta)}_{\text{cost}} + \underbrace{5v}_{\text{compliance}} + \underbrace{\text{round}(p \cdot P_{\text{ml}})}_{\text{ML code-risk}}, \quad (2.2)$$

where $P_{\text{ml}} = 20$ is the maximum contribution of the machine-learning component at $p = 1$. The cost contribution $c(\Delta)$ is banded: a high band applies above a high monthly-cost threshold and a medium band above a medium threshold, with the thresholds and their points being configurable. Banding, rather than a continuous cost term, keeps the score stable against small estimation noise.

The decision function maps S to one of three outcomes using the thresholds in Table 2.5:

$$\text{decision}(S) = \begin{cases} \text{pass} & \text{if } S \leq 20, \\ \text{review} & \text{if } 20 < S \leq 80, \\ \text{reject} & \text{if } S > 80. \end{cases} \quad (2.3)$$

Figure 2.6 summarises the engine: the four component sub-scores are summed and compared against the two thresholds to select the decision.

Table 2.5: Decision thresholds and their operational meaning.

Decision	Score band	Action
Pass	≤ 20	Auto-deploy; no human review required.
Review	21–80	Manual approval required before deployment.
Reject	> 80	Do not deploy; regenerate (if generating) or return to the author.

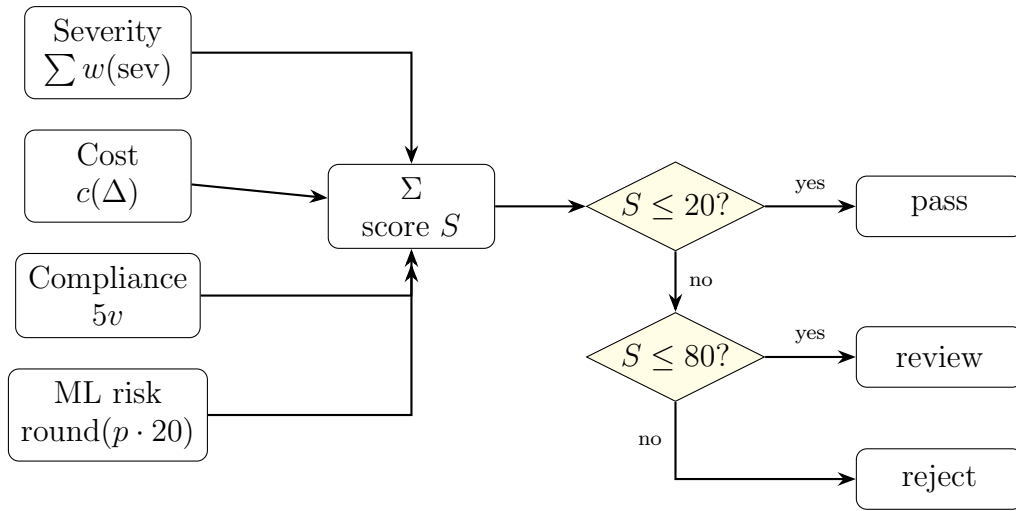


Figure 2.6: The additive risk-scoring engine: four orthogonal sub-scores are summed and the total is compared against two fixed thresholds to yield the decision.

2.3.4 Machine-learning code-risk model

The machine-learning component predicts whether generated Python code is likely to be insecure, supplying the probability p used in Equation 2.2. Features are extracted from two static analysers — a security linter and a pattern-matching analyser — applied *per file*: the counts of findings grouped by severity form the feature vector. A logistic-regression classifier trained on a labelled dataset of secure and insecure code produces a probability of insecurity per file, and the gate takes the worst (maximum)

probability across the generated files, so that a single risky file cannot be masked by many benign ones.

$$p = \max_{k \in \text{files}} \sigma(\mathbf{w}^\top \mathbf{x}_k + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (2.4)$$

where $\mathbf{x}_k$ is the per-file severity-count feature vector, and $\mathbf{w}, b$ are the learned logistic-regression parameters. The decisive design constraint is *inference/training parity*: feature extraction at inference time uses the same tools, the same severity mapping, and the same per-file scanning as during training, so the inputs seen by the classifier match its training distribution. When the model artifacts or the analyser binaries are unavailable, the component degrades to a “skipped” status and contributes zero points, consistent with the graceful-degradation contract of the overall system.

2.3.5 Deduplication and explainability

Two properties make the score trustworthy. First, *deduplication* guarantees that no single underlying issue is counted twice: findings are keyed by (`resource_id`, `template`, `category`), and colliding findings are merged, keeping the highest severity and unioning the source labels. Without this step, an issue reported by three tools would triple its severity contribution and could push an otherwise acceptable change over a threshold.

Second, *explainability* follows directly from the additive form of Equation 2.2: because the score is a sum of clearly labelled components, the gate report includes a per-component breakdown that attributes exactly how many points came from severity, cost, compliance, and the machine-learning model. An operator reviewing a *review* or *reject* decision can therefore see *why* the score landed where it did and which component to remediate, rather than being handed an opaque number.

2.3.6 Worked example

Consider a change whose deduplicated findings include one high-severity open administrative port and two medium-severity issues, whose estimated monthly cost increase falls in the medium band (contributing 10 points), which trips one non-compliant live rule, and whose generated code the machine-learning model scores at $p = 0$ (secure). Applying Equation 2.2:

$$\begin{aligned} S &= \underbrace{(10 + 5 + 5)}_{\text{severity}} + \underbrace{10}_{\text{cost}} + \underbrace{5 \times 1}_{\text{compliance}} + \underbrace{\text{round}(0 \times 20)}_{\text{ML}} \\ &= 20 + 10 + 5 + 0 = 35. \end{aligned}$$

Since $20 < 35 \leq 80$, Equation 2.3 yields *review*: the change is not auto-deployed but is instead routed to a human approver, whose approval (or rejection) is recorded in the audit trail. The per-component breakdown (severity = 20, cost = 10, compliance = 5, ML = 0) tells the approver that the open administrative port and the cost increase are the dominant contributors, directing remediation effort precisely.

2.4 Implementation

This section describes how the proposed methodology is realised in software. The system is implemented in Python and is exposed both as a set of command-line entry points and as a browser-based operator console. The implementation follows the module boundaries introduced in Section 2.1: generation, evaluation, execution, orchestration, monitoring, and the user interface.

2.4.1 Technology stack and repository layout

The public side is defined with the AWS Cloud Development Kit for Python and synthesized to CloudFormation; the private side is defined with Ansible playbooks. Static analysis relies on established open-source scanners, cost estimation on a cost-analysis tool, and live compliance on the cloud provider’s configuration service. The operator console is built with Streamlit. Table 2.6 lists the directories that implement each concern.

Table 2.6: Repository layout and the concern each directory implements.

Directory	Contents
AIgen/	Provider backends and the CDK regeneration loop (<code>run_cdk_regen.py</code>).
Eval/	The gate (<code>iac_security_gate.py</code>) and the scanner adapters under <code>Eval/scanners/</code> .
ExecComponent/	The subprocess execution helper (<code>exec_code.py</code>).
pipeline/	Orchestration, credentials, and Phase 4 emitters.
Monitor/	CloudTrail, dashboards, the operational-loop function, and SSM registration.
scripts/	<code>run_cdk_pipeline.py</code> and <code>run_hybrid_pipeline.py</code> .
ui/	The Streamlit console and the multi-project registry.
RiskScoringCitation/	Dataset preparation and model training.

Figure 2.7 shows how these directories depend on one another at run-time. The command-line entry points in `scripts/` sit at the top; they call into `pipeline/` for orchestration, which in turn drives generation (`AIgen/`), evaluation (`Eval/`), and monitoring (`Monitor/`). Every directory that runs an external process funnels through the single execution helper in `ExecComponent/`, and the operator console in `ui/` reuses the same entry

points rather than duplicating logic.

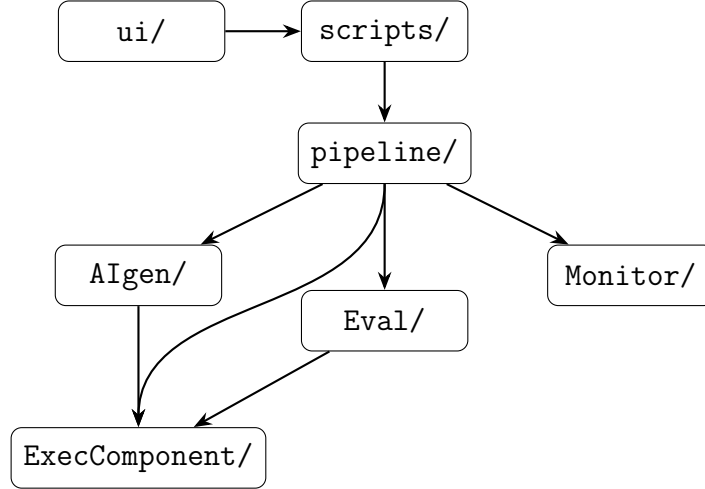


Figure 2.7: Runtime module dependencies. Entry points drive orchestration, which drives generation, evaluation, and monitoring; all external processes pass through the single execution helper.

2.4.2 Generation layer

Two interchangeable backends implement the generation contract behind a common interface, so that the choice of provider is a configuration flag rather than a code change. Both expose the same functions to call the model, extract the code from the response, and save the result, and both centralise their default model identifiers while allowing an explicit override. Provider-specific error handling (for example, quota and throttling detection) is isolated behind the provider boundary, and credentials are read from explicit arguments first and then from the environment. The re-generation loop described in Algorithm 1 orchestrates these backends and persists the per-attempt artifacts.

2.4.3 Security gate and scanner adapters

The gate is implemented in `iac_security_gate.py`, which collects the synthesized templates, runs the heuristic checks and the scanner adapters,

deduplicates the findings, computes the score of Equation 2.2, applies the decision function of Equation 2.3, and writes the gate report. Each scanner lives in its own adapter and returns a dictionary with at least a status and a message; no adapter ever raises, so a missing tool degrades to a well-defined status while the gate proceeds. Table 2.7 lists the adapters.

Table 2.7: Scanner adapters and the signal each contributes.

Adapter	Signal
<code>checkov_adapter.py</code>	Policy-as-code findings on the synthesized templates.
<code>cfn_lint_adapter.py</code>	CloudFormation best-practice and validity findings.
<code>infracost_adapter.py</code>	Estimated monthly cost, used as the cost dimension.
<code>aws_config_adapter.py</code>	Count of non-compliant compliance rules.
<code>ansible_lint_adapter.py</code>	Findings on the on-premises playbooks.
<code>secret_scan_adapter.py</code>	Regular-expression secret detection (no external tool).
<code>ml_risk_adapter.py</code>	Logistic-regression probability of insecurity, converted to points.

An abbreviated gate report is shown in Listing 2.1. The `components` object exposes the per-dimension contribution to the score, which makes the decision explainable, and the `scanner_status` object records which tools ran.

```

1 {
2   "run_id": "cdk_20260614T093934Z",
3   "timestamp": "2026-06-14T09:39:42+00:00",
4   "score": 50,
5   "decision": "review",
6   "message": "Manual review required before deployment.",
7   "components": {"severity": 30, "cost": 10, "aws_config": 10, "ml_risk":
8     0},
9   "findings": [
    {"severity": "high", "source": "checkov", "category": "sg_ssh_open",

```

```

10     "resource_id": "MySG", "template": "App.template.json",
11     "message": "Security group allows ingress on port 22 from 0.0.0.0/0"}
12 ],
13 "scanner_status": {"checkov": "ok", "cfn_lint": "ok",
14                    "infracost": "ok", "aws_config": "ok", "ml_risk": "ok"
15 }

```

Listing 2.1: Abbreviated gate report.

2.4.4 Orchestration and governance

Two command-line entry points drive the pipeline. The CDK-only entry point, `run_cdk_pipeline.py`, orchestrates bootstrap, synthesis, gating, diff, and deployment for the public side, and can optionally invoke the regeneration loop on a reject. The hybrid entry point, `run_hybrid_pipeline.py`, runs the same gate, risk-scoring, deployment, and observability workflow over both a bring-your-own CDK directory and a bring-your-own Ansible directory — with no code-generation step — gating each side independently with the same engine and writing a combined report. The orchestration routes all external commands through the execution backbone and honours the deployment-governance rules of Section 2.2.6. Listing 2.2 shows representative invocations.

```

1 # CDK-only: synth, gate, and deploy if the gate allows
2 python scripts/run_cdk_pipeline.py --project-dir GeneratedCDK --deploy
3
4 # Hybrid: gate both sides; scope the Ansible scan to files changed since
   HEAD~1
5 python scripts/run_hybrid_pipeline.py \
6     --cdk-path examples/hybrid-demo/cdk \
7     --ansible-path examples/hybrid-demo/ansible \
8     --base-ref HEAD~1
9
10 # Read-only status of every gated target
11 python scripts/run_hybrid_pipeline.py --query-status

```

Listing 2.2: Representative pipeline invocations.

On the hybrid side, the Ansible scan can be scoped to the files changed since a given git reference, which limits analysis to the modified plays. The decision returned by each entry point is surfaced as a distinct process return code, so that both the console and any external continuous-integration system can react to *pass*, *review*, and *reject* without parsing free text.

Figure 2.8 traces the hybrid pipeline as a sequence of interactions between the orchestrator, the gate, the cloud and on-premises targets, and the monitoring layer, showing where the decision gates the deployment step on each side independently.

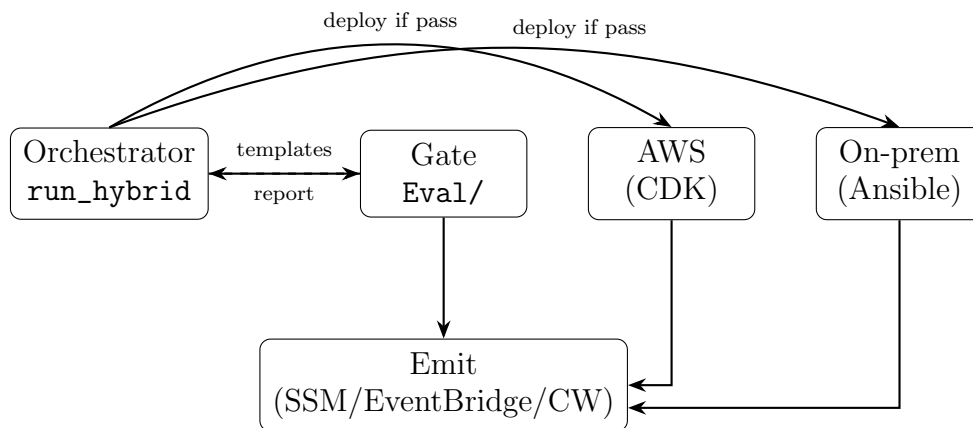


Figure 2.8: Hybrid pipeline interaction sequence. The orchestrator gates both sides with the same engine, deploys each side only on an accepting decision, and emits telemetry.

2.4.5 Execution backbone

All external commands (CDK, Ansible, and the scanners) run through a single subprocess helper in `exec_code.py`. The helper merges standard output and standard error to preserve the ordering of events, optionally streams output line by line for live rendering in the console, and returns the stable `{return_code, output}` contract that every caller relies on. Concentrating process execution in one place keeps the output-capture and error-propagation behaviour consistent across the whole system.

2.4.6 Phase 4 observability

The monitoring layer implements three complementary observability concerns. Gate metrics (score, decision, and the component breakdown) are published to a custom CloudWatch namespace; compliance drift is watched by the configuration service and reacted to by an event-driven function; and an audit trail of API activity is captured by CloudTrail. A dashboard presents the risk score and the decision per target for both the public and the private side, and the operational-loop function reacts to drift and rollback events to enable automated remediation. The event shapes and parameter layout emitted by the orchestration layer are consumed unchanged by this layer, which keeps the two decoupled.

2.4.7 Operator console and multi-project registry

The Streamlit console is a thin orchestration layer over the command-line entry points: every privileged action is delegated to a subprocess with credentials injected through the environment, never as command-line arguments. The console presents the workflow as a sequence of tabs — settings, generation, gate review, plan/diff, deploy, and monitor — and renders the gate report, the decision, and the live monitoring reads. A multi-project registry allows several independent projects to coexist: the default project keeps a flat log layout for backward compatibility, while any other project owns a separate artifact directory so that gate reports, approvals, rejections, and monitoring history never mix between projects. The active project's log directory is exported to the subprocesses through an environment variable, which is how per-project isolation is enforced end to end.

Chapter 3

Hướng dẫn sử dụng template

3.1 Trích dẫn tài liệu

Dùng lệnh `\cite` để trích dẫn một hoặc nhiều tài liệu tham khảo. Tài liệu tham khảo có thể là trang web [1], [2], bài báo khoa học [3], sách [4], [5], [6], bài tạp chí [7] hoặc các nguồn tham khảo khác. Lưu ý khi trích dẫn tài liệu tham khảo, cần viết câu sao cho bỏ phần trong cặp ngoặc vuông đi thì câu vẫn đầy đủ ý nghĩa. Ví dụ, thay vì viết “Nghiên cứu [7] chỉ ra rằng ... ” thì nên viết “Nghiên cứu của Zhang [7] chỉ ra rằng ...”. Một ví dụ khác, thay vì viết “... như trong công trình nghiên cứu [3].” thì nên viết “... như trong công trình nghiên cứu của Cavnar và Trenkle [3].”

3.2 Chèn mã nguồn

Để chèn mã nguồn, cần dùng package listings [1]:

```
1 \usepackage{listings}
```

Mã nguồn có thể được chèn trực tiếp như sau:

```
1 print "Hello , World!"
```

hoặc chèn thông qua tập tin chứa mã nguồn trong thư mục *SourceCode* như sau:

```
1 #include <iostream.h>
```

```

2
3 main()
4 {
5     for (;;)
6     {
7         cout << "Hello World! ";
8     }
9 }

```

Để chèn mã giả, cần dùng package algorithm [8]:

```

1 \usepackage{algorithm}

```

Có thể chèn mã giả vào như sau:

Algorithm 2 My algorithm

```

1: procedure MYPROCEDURE
2:    $stringlen \leftarrow \text{length of } string$ 
3:    $i \leftarrow patlen$ 
4: top:
5:   if  $i > stringlen$  then return false
6:    $j \leftarrow patlen$ 
7: loop:
8:   if  $string(i) = path(j)$  then
9:      $j \leftarrow j - 1.$ 
10:     $i \leftarrow i - 1.$ 
11:    goto loop.
12:   close;
13:    $i \leftarrow i + \max(delta_1(string(i)), delta_2(j)).$ 
14:   goto top.

```

3.3 Hình ảnh

Để chèn hình ảnh, cần dùng package graphicx [9]:

```

1 \usepackage{graphicx}

```

Hình 3.1, hình 3.2 là một số ví dụ về chèn hình ảnh.



Figure 3.1: Hình ví dụ 1



Figure 3.2: Hình ví dụ 2

3.4 Bảng biểu

Để tạo bảng biểu, tham khảo thêm tại sharelatex.com [10]. Bảng 3.1 là một ví dụ về bảng. Ngoài ra, có một số tool online ¹ có thể được dùng để tạo bảng biểu một cách trực quan.

3.5 Công thức

Công thức có thể chèn vào trong cùng một dòng như $\sqrt{a^2 + b^2}$ hoặc nằm trên dòng riêng như công thức 3.1.

$$x = a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{a_3 + a_4}}} \quad (3.1)$$

¹Như trang <http://www.tablesgenerator.com/>

Table 3.1: Bảng ví dụ 1

Country List			
Country Name or Area Name	ISO ALPHA 2 Code	ISO ALPHA 3 Code	ISO numeric Code
Afghanistan	AF	AFG	004
Aland Islands	AX	ALA	248
Albania	AL	ALB	008
Algeria	DZ	DZA	012
American Samoa	AS	ASM	016
Andorra	AD	AND	020
Angola	AO	AGO	024

Danh mục công trình của tác giả

1. Tạp chí ABC
2. Tạp chí XYZ

Tài liệu tham khảo

Tiếng Việt

- [2] D.-H. Trường Đại học Khoa học Tự nhiên. “Quy định và hướng dẫn thực hiện luận văn thạc sĩ,” Accessed: Jun. 6, 2015. [Online]. Available: http://www.hcmus.edu.vn/index.php?option=com_content&task=blogcategory&id=142&Itemid=506
- [5] D. Điền, *Xử lý ngôn ngữ tự nhiên*. Nhà xuất bản Đại học Quốc gia TP.HCM, 2006.
- [6] D. Q. Ban and H. V. Thung, *Ngữ pháp tiếng Việt*. Nhà xuất bản Giáo dục Việt Nam, 2006.

Tiếng Anh

- [1] Online. “Latex/source code listings,” Accessed: Jun. 6, 2015. [Online]. Available: http://en.wikibooks.org/wiki/LaTeX/Source_Code_Listings
- [3] Cavnar, W. B. and Trenkle, J. M., “N-gram-based text categorization,” in *In Proceedings of SDAIR-94, 3rd Annual Symposium on Document Analysis and Information Retrieval*, 1994, pp. 161–175.
- [4] Knuth, D. E., *The T_EXbook*. Addison-Wesley, 1984.

- [7] Zhang, K. and Shasha, D., “Simple fast algorithms for the editing distance between trees and related problems,” *SIAM Journal on Computing*, Volume 18 Issue 6, pp. 1245–1262, 1989.
- [8] Online. “Latex/algorithms,” Accessed: Jun. 6, 2015. [Online]. Available: <http://en.wikibooks.org/wiki/LaTeX/Algorithms>
- [9] Online. “Latex/floats, figures and captions,” Accessed: Jun. 6, 2015. [Online]. Available: http://en.wikibooks.org/wiki/LaTeX/Floats,_Figures_and_Captions
- [10] Online. “Latex/tables,” Accessed: Jun. 6, 2015. [Online]. Available: <http://en.wikibooks.org/wiki/LaTeX/Tables>

Appendix A

Ngữ pháp tiếng Việt

Đây là phụ lục.

Appendix B

Ngữ pháp tiếng Nôm

Đây là phụ lục 2.